

Vizja Tech E-commerce: Technical Documentation

Technical Architecture & System Design

1. Project Overview

The **Vizja Tech E-commerce Platform** is a robust, full-stack web application developed using the Django framework. The project is designed to provide a seamless online shopping experience, featuring a dynamic product catalog, secure user authentication, and a structured order management system. The application follows the Model-Template-View (MTV) architectural pattern to ensure a clean separation of concerns.

2. System Architecture

The system is built on a modular architecture where each core functionality is isolated into specific Django apps:

Accounts Module (accounts/): Manages user lifecycle, including custom user registration, secure login sessions, and profile management.

Store Module (store/): The primary engine for product display, search functionality, and individual product detail rendering.

Category Module (category/): Provides a hierarchical organization for products, allowing for efficient database queries and navigation.

Cart & Order Modules (carts/, orders/): Manages the transition from product selection to final purchase, including session-based cart persistence and order record generation.

3. Technology Stack

Backend Framework: Python 3.x / Django

Database: SQLite3 (Relational)

Frontend: Django Template Language (DTL), HTML5, CSS3, JavaScript

Asset Management: Django Static Files & Media Root (for product images)

4. Data Models and Schema

The database is structured to maintain referential integrity across the following entities:

Account: Stores hashed credentials and user metadata.

Product: Contains pricing, stock levels, descriptions, and foreign keys to Categories.

CartItem: Links products to specific user sessions or accounts.

Order: Captures a snapshot of products, prices, and shipping details at the time of purchase.

Operations, Deployment & Maintenance

1. Installation and Environment Setup

To maintain or deploy the project, follow the standardized environment configuration:

Virtual Environment: Initialize a Python virtual environment to isolate dependencies.

Dependency Installation: Install required packages via `pip install -r requirements.txt`.

Database Initialization: Run `python manage.py migrate` to apply the schema to the SQLite database.

Superuser Creation: Use `python manage.py createsuperuser` to access the administrative dashboard.

2. Administrative Management

The project includes a pre-configured Django Admin Interface accessible at the `/admin` path.

This interface is the primary tool for:

Inventory Updates: Adding, editing, or deleting products and categories.

Order Tracking: Reviewing customer orders and updating fulfillment statuses.

User Oversight: Managing user permissions and account security.

3. Security & Scalability Protocols

Data Security: All passwords are encrypted using PBKDF2 with a SHA256 hash. Cross-Site Request Forgery (CSRF) protection is enabled on all transaction forms.

Media Handling: Product images are stored in the `media/` directory. For production scaling, it is recommended to integrate an Amazon S3 bucket for static asset delivery.

Future Enhancements: The modular design allows for the future integration of third-party payment gateways (e.g., Stripe) and automated email notifications for order confirmations.